

```
import pandas as pd
import sqlite3
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from google.colab import files
uploaded = files.upload()
```

[Choose files](#) Book 4.xlsx
Book 4.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 53440 bytes, last modified: 05/05/2026 - 100% done
Saving Book 4.xlsx to Book 4 (1).xlsx

```
df = pd.read_csv("student_performance_prediction.csv")
df.head()
```

	Student ID	Study Hours per Week	Attendance Rate	Previous Grades	Participation in Extracurricular Activities	Parent Education Level	Passed	
0	S00001	12.5	NaN	75.0	Yes	Master	Yes	
1	S00002	9.3	95.3	60.6	No	High School	No	
2	S00003	13.2	NaN	64.0	No	Associate	No	
3	S00004	17.6	76.8	62.4	Yes	Bachelor	No	
4	S00005	8.8	89.3	72.7	No	Master	No	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df.isnull().sum()
```

	0
Student ID	0
Study Hours per Week	1995
Attendance Rate	1992
Previous Grades	1994
Participation in Extracurricular Activities	2000
Parent Education Level	2000
Passed	2000

dtype: int64

```
df = df.dropna()
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df = df.drop_duplicates()
```

```
import sqlite3

conn = sqlite3.connect("student.db")
df.to_sql("students", conn, if_exists="replace", index=False)
```

29446

```
query = """
SELECT `Participation in Extracurricular Activities`, COUNT(*) as total_students
FROM students
GROUP BY `Participation in Extracurricular Activities`
"""
pd.read_sql(query, conn)
```

1 to 2 of 2 entries Filter ?

index	Participation in Extracurricular Activities	total_students
0	No	14739
1	Yes	14707

Show 25 per pageLike what you see? Visit the [data table notebook](#) to learn more about interactive tables.

```
query = """
SELECT `Participation in Extracurricular Activities`,
       AVG(CASE WHEN Passed='Yes' THEN 1 ELSE 0 END) as pass_rate
FROM students
GROUP BY `Participation in Extracurricular Activities`
"""
pd.read_sql(query, conn)
```

Participation in Extracurricular Activities	pass_rate
0	No 0.496167
1	Yes 0.501598

```
# Study level
df["study_level"] = pd.cut(df["Study Hours per Week"],
                           bins=[0,8,15,50],
                           labels=["Low","Medium","High"])

# Attendance level
df["attendance_level"] = pd.cut(df["Attendance Rate"],
                                bins=[0,70,85,100],
                                labels=["Low","Medium","High"])

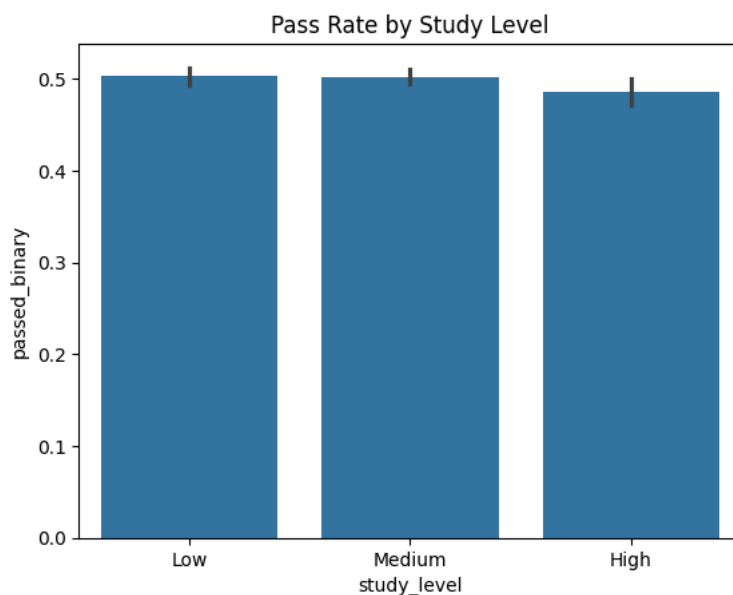
# Convert passed to numeric
df["passed_binary"] = df["Passed"].map({"Yes":1,"No":0})
```

df.corr(numeric_only=True)

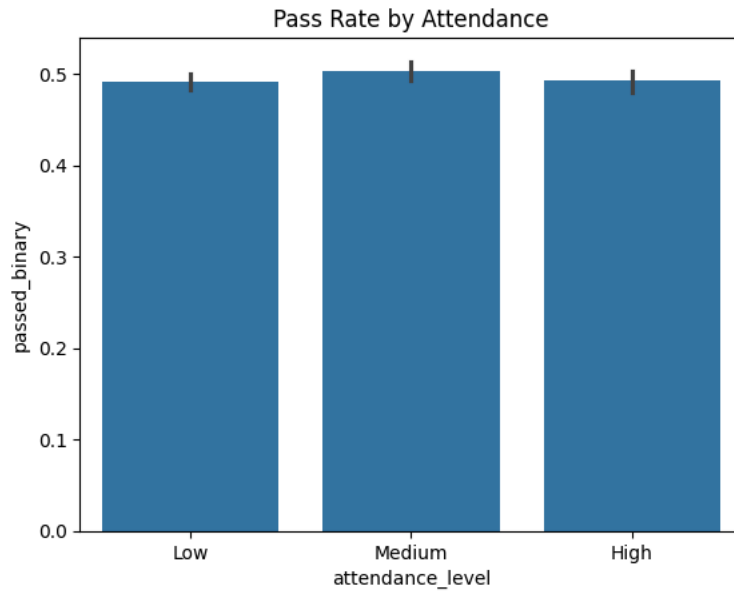
	Study Hours per Week	Attendance Rate	Previous Grades	passed_binary
Study Hours per Week	1.000000	0.006096	0.002378	-0.008987
Attendance Rate	0.006096	1.000000	-0.004800	0.011095
Previous Grades	0.002378	-0.004800	1.000000	0.004640
passed_binary	-0.008987	0.011095	0.004640	1.000000

```
import seaborn as sns
import matplotlib.pyplot as plt

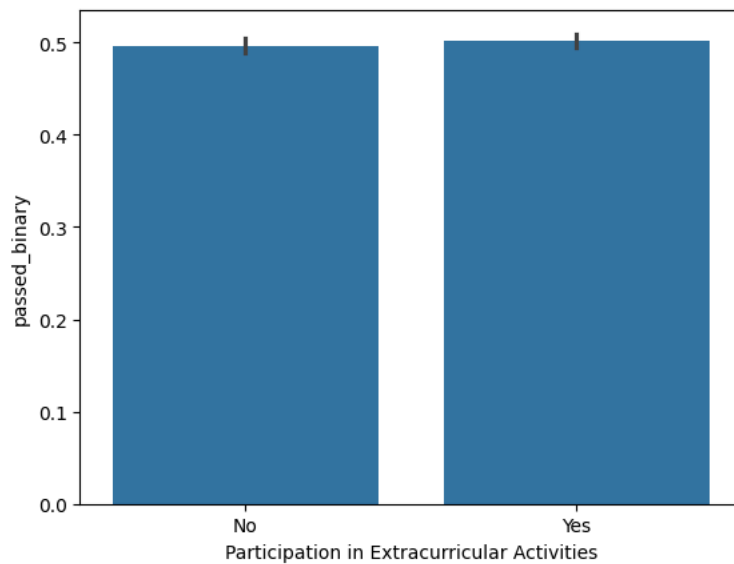
sns.barplot(x="study_level", y="passed_binary", data=df)
plt.title("Pass Rate by Study Level")
plt.show()
```



```
sns.barplot(x="attendance_level", y="passed_binary", data=df)
plt.title("Pass Rate by Attendance")
plt.show()
```



```
sns.barplot(x="Participation in Extracurricular Activities", y="passed_binary", data=df)
plt.show()
```



```
import statsmodels.api as sm

X = df[["Study Hours per Week", "Attendance Rate", "Previous Grades"]]
y = df["passed_binary"]

X = sm.add_constant(X)

model = sm.OLS(y, X).fit()
print(model.summary())
```

```
=====
                        OLS Regression Results
=====
```

Dep. Variable:	passed_binary	R-squared:	0.000
Model:	OLS	Adj. R-squared:	0.000
Method:	Least Squares	F-statistic:	2.231
Date:	Tue, 05 May 2026	Prob (F-statistic):	0.0823
Time:	12:30:20	Log-Likelihood:	-21368.
No. Observations:	29446	AIC:	4.274e+04
Df Residuals:	29442	BIC:	4.278e+04
Df Model:	3		
Covariance Type:	nonrobust		

```
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	0.4779	0.017	28.083	0.000	0.445	0.511
Study Hours per Week	-0.0009	0.001	-1.556	0.120	-0.002	0.000
Attendance Rate	0.0003	0.000	1.917	0.055	-6.11e-06	0.001

```
=====
```

```
Previous Grades      0.0001      0.000      0.809      0.418      -0.000      0.000
=====
Omnibus:             100661.443      Durbin-Watson:             1.990
Prob(Omnibus):        0.000      Jarque-Bera (JB):         4903.208
Skew:                 0.004      Prob(JB):                 0.00
Kurtosis:             1.001      Cond. No.                 596.
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
df.to_csv("cleaned_student_data.csv", index=False)
```

```
from google.colab import files
files.download("cleaned_student_data.csv")
```

Start coding or [generate](#) with AI.

